Lecture 6

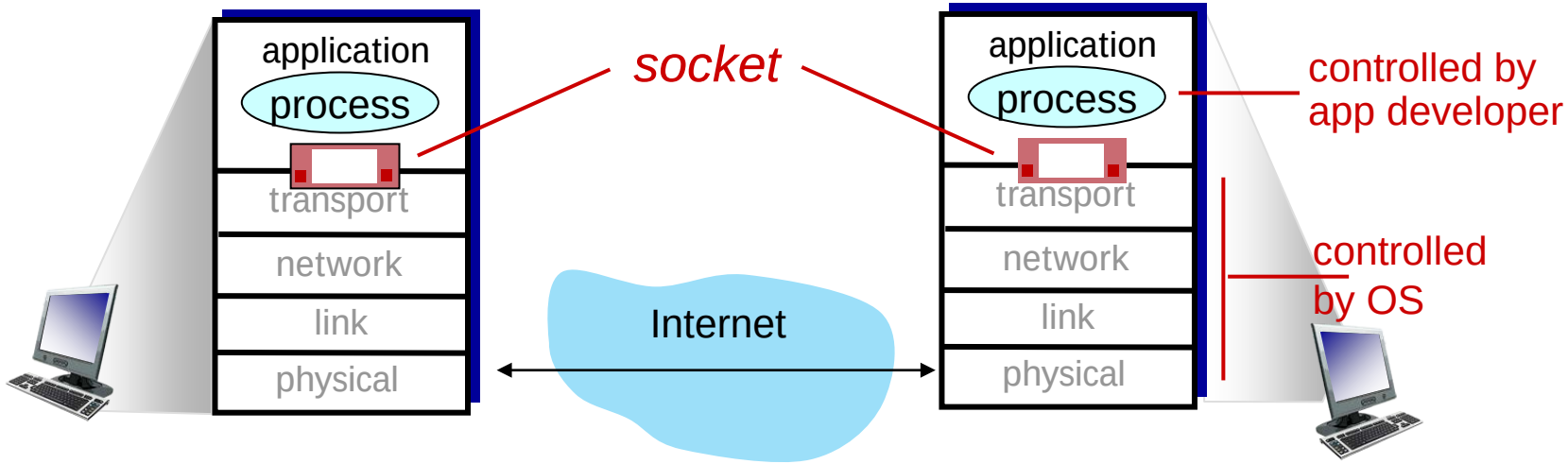
- Network programming and sockets
- Transport Layer services
- Web and HTTP

Network Programming

- We talked about: Internet as a service infrastructure
- To use the network, we write programs that:
  - run on (different) end systems
  - communicate over network
  - e.g., web server software communicates with browser software
- Users do not need to write software for network-core devices
  - network-core devices do not run user applications
  - applications on end systems allows for rapid app development

Sockets

- process sends/receives messages to/from its socket
- Socket analogous to door
  - sending process shoves message out the door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process



Analogy Between a Socket and a Door

End hosts ➔ Houses; Sockets ➔ Doors; Process ➔ Letter writers

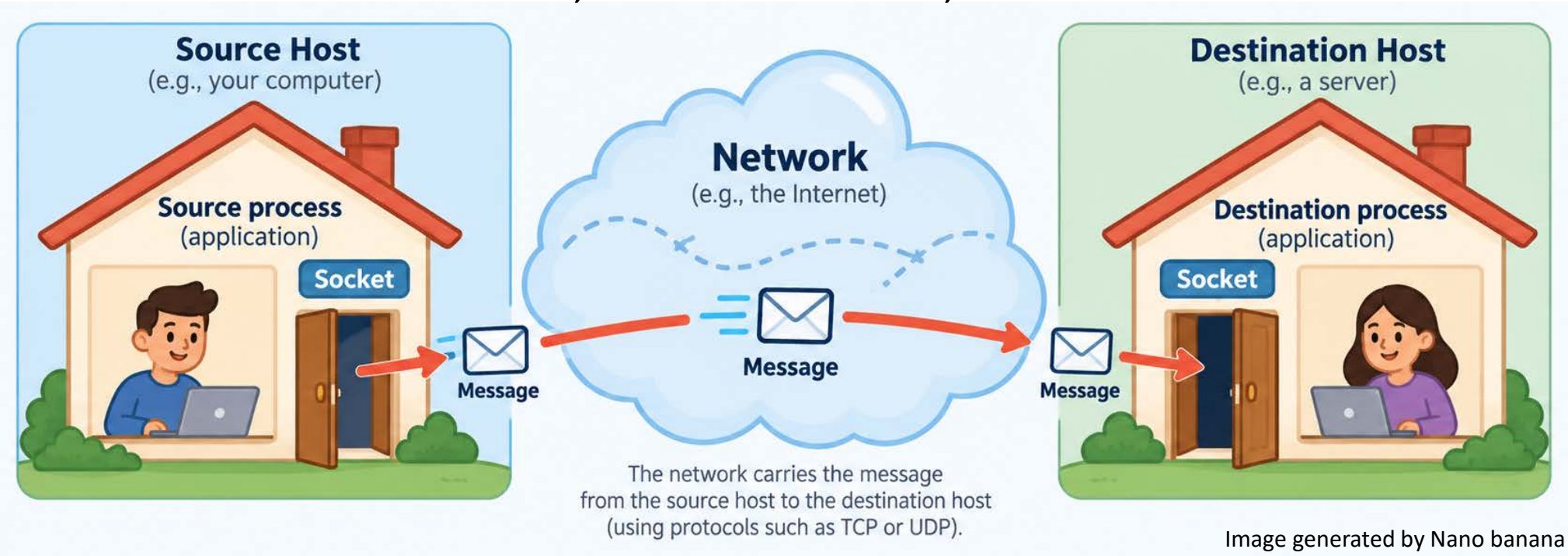


Image generated by Nano banana

A socket is the interface (“door”) through which a process sends and receives messages across the network.

Q: What happens if there are multiple process? Need multiple sockets/doors?

Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- *Q:* does IP address of host suffice for identifying the process?
  - *A:* no, *many* processes can be running on the same host

- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
  - HTTP server: 80
  - mail server: 25
- Reserved assigned numbers specified via RFCs (e.g., https://datatracker.ietf.org/doc/html/rfc1340)

Addressing processes

PORT	SERVICE	PROTOCOL
20/21	File Transfer Protocol (FTP) data/FTP command	TCP
22	SSH/Secure File Transfer Protocol (SFTP)	TCP
23	Telnet	TCP
25	Simple Mail Transfer Protocol (SMTP)	TCP
53	Domain Name Service (DNS)	UDP
67/68	Dynamic Host Configuration Protocol (DHCP)	UDP
69	Trivial File Transfer Protocol (TFTP)	UDP
79	Finger protocol (user information)	TCP
80	Hypertext Transfer Protocol (HTTP)	TCP
88	Kerberos	UDP
110	Post Office Protocol version 3 (POP3)	TCP
111	Sun Microsystems Remote Procedure Call (SUNRPC)	TCP/UDP
123	Network Time Protocol (NTP)	UDP
135	Microsoft Remote Procedure Call (MS RPC)	TCP/UDP
139	NetBIOS Session	TCP/UDP
143	Internet Message Access Protocol (IMAP)	TCP
156	Structured Query Language (SQL) service	TCP/UDP
161	Simple Network Management Protocol (SNMP)	UDP
162	SNMP Trap	UDP
179	Border Gateway Protocol (BGP)	TCP/UDP
443	HTTP Secure (HTTPS)	TCP

- There are 65,536 port numbers (2^16)
- Well-known ports (0 to 1023)
  - reserved for common, standard TCP/IP applications and services
- Registered ports (1024 to 49,151)
  - Assigned to specific applications and services of a vendor
- Dynamic ports (49,152 to 65,535)

Performance Requirements

- Reliability: audio apps can tolerate some loss while file transfers usually do not.
- Interactive apps require low delay
- Multimedia apps require a minimum throughput to work
  - Elastic apps: Adapt to provided throughput
- Security: encryption, data integrity

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec

Internet Transport Services

- Transmission Control Protocol (TCP)
- User Datagram Protocol (UDP)

TCP:

- *Connection-oriented:* setup required between client and server processes
- *Reliable packet transport:* no-loss, in-order delivery
- *Flow control:* sender won't overwhelm receiver
- Congestion control: sender won't overwhelm network
- *No guarantees on delay and throughput*

UDP :

- *unreliable data transfer* between sending and receiving process
- *does not provide:* reliability, timing, throughput guarantee, security, or connection setup.
- "Quick and dirty"

Q: why bother? *Why* is there a UDP?

Securing TCP

Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

TLS implemented in application layer

- More precisely, between the application and transport layers (session layer)
- Apps use TLS libraries, that use TCP in turn
- cleartext sent into "socket"; then traverse Internet *encrypted*

Question 1

- A network application process is identified uniquely by:
  A. IP address
  B. IP address and port number
  C. IP address, port number, and MAC address
  D. Domain name

Question 2: True/False

Select the statements that are true.

A. TCP and UDP are Transport Layer protocols
B. TCP and UDP are Application Layer protocols.
C. Sockets are interfaces between TCP and UDP protocols.
D. Sockets are interfaces between any application layer protocol and its underlying transport layer protocol.

Application layer: Outline

- Overview
- Examples of Internet Applications
  - Web and HTTP
  - E-mail, SMTP, IMAP
  - The Domain Name System DNS
  - P2P applications
  - video streaming and content distribution networks
- Socket Programming

The Web

First, a quick review…

- web page consists of *objects,* each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,…
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL,* e.g.,

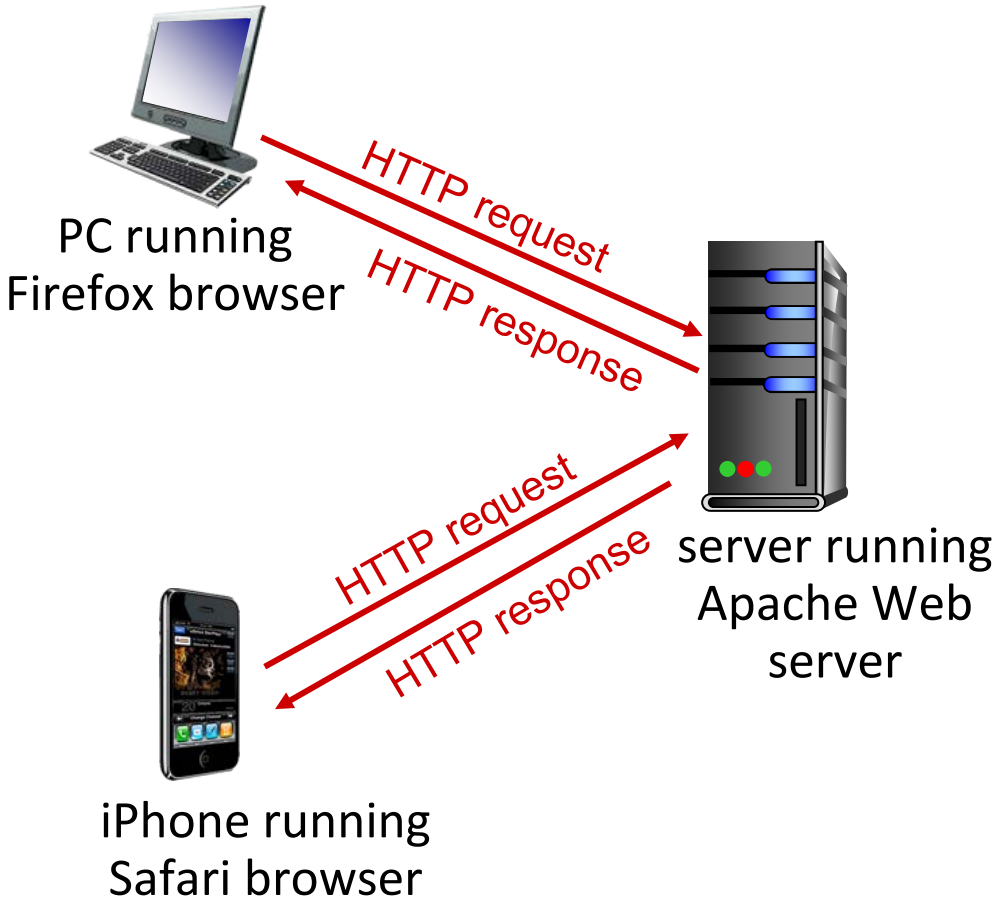
`www.someschool.edu/someDept/pic.gif`

host name | path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model:
  - *client:* browser that requests, receives, (using HTTP protocol) and "displays" Web objects
  - *server:* Web server sends (using HTTP protocol) objects in response to requests
- http 1.0: RFC 1945
- http 1.1: RFC 2616 ; New RFC 7230
- http 2 : RFC 7540



HTTP overview (continued)

HTTP 1 and 2 use TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

Non-persistent HTTP: example

User enters URL: **`www.someSchool.edu/someDepartment/home.index`**
(containing text, references to 10 jpeg images)



1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80



1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80 "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

Non-persistent HTTP: example (cont.)

User enters URL: **`www.someSchool.edu/someDepartment/home.index`**
(containing text, references to 10 jpeg images)





4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

time

HTTP connections: two types

Non-persistent HTTP

- HTTP/1.0
  - TCP connection opened
  - server parses request, responds, and closes TCP connection
- 2 RTTs to fetch each object
- downloading multiple objects requires multiple connections

Persistent HTTP

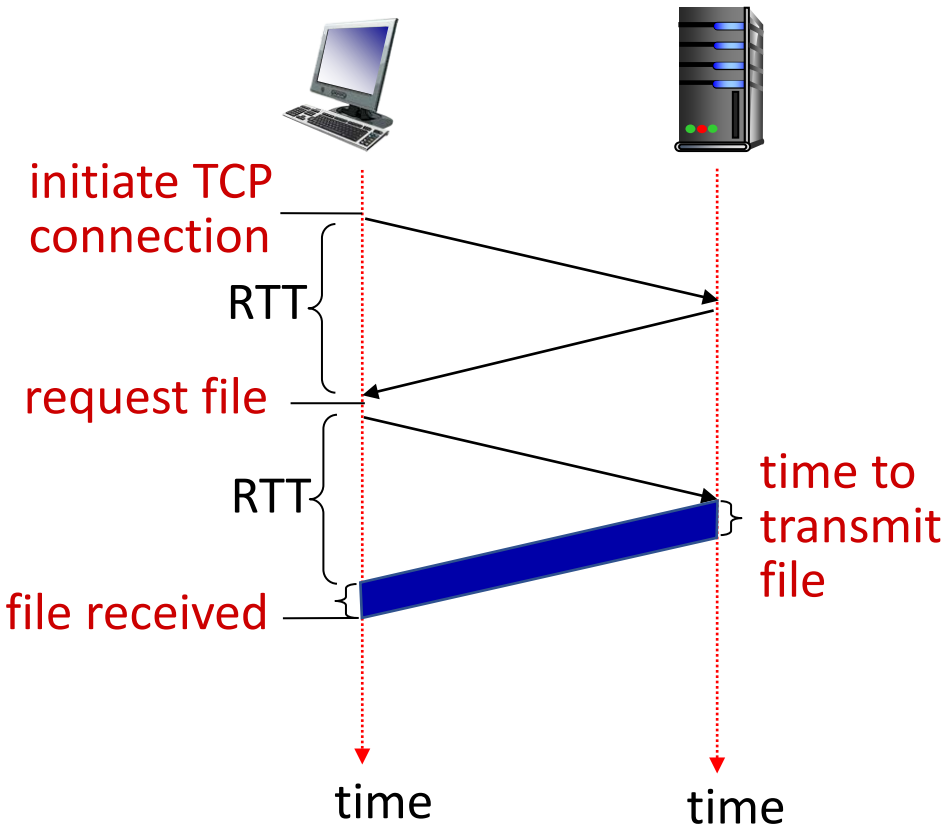
- Default for HTTP/1.1
- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



Non-persistent HTTP response time = 2RTT+ file transmission time

Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues:

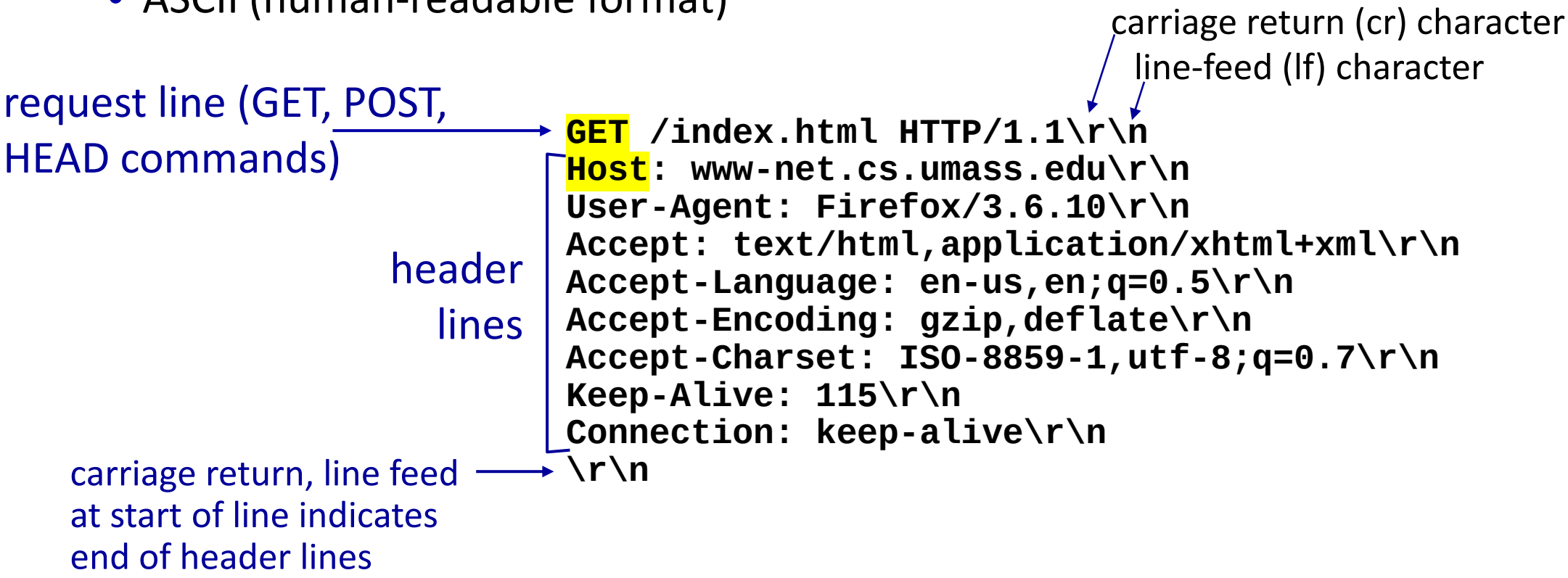
- requires 2 RTTs per object
- OS overhead for *each* TCP connection

Persistent HTTP (HTTP1.1):

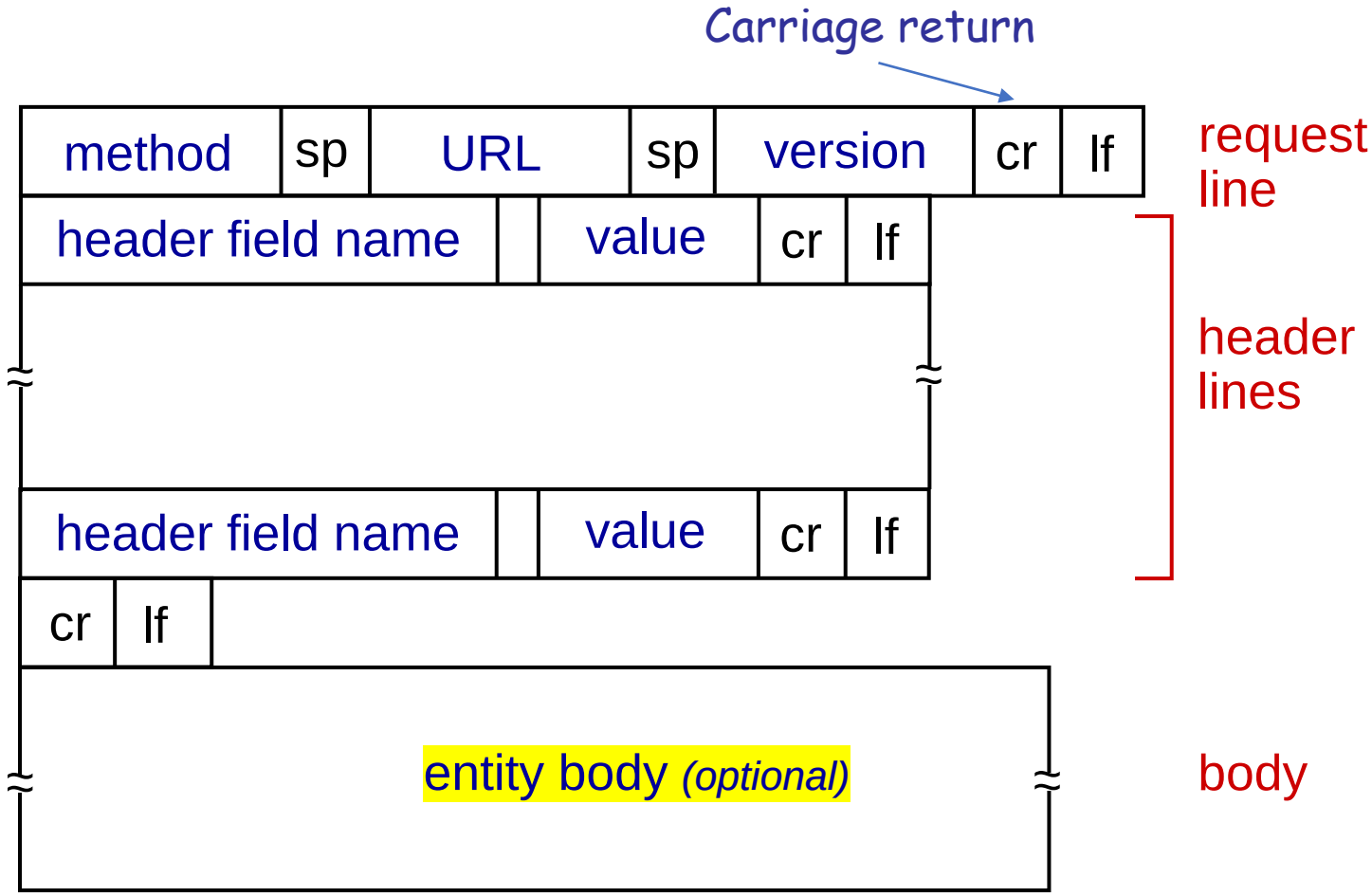
- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)



HTTP request message: general format



entity body: used with "state-changing HTTP methods", such as **POST or PUT.**

Popular HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

```
www.somesite.com/animalsearch?monkeys&banana
```

HEAD method:

- ***requests headers (only)*** that would be returned *if* specified URL were requested with an HTTP GET method.

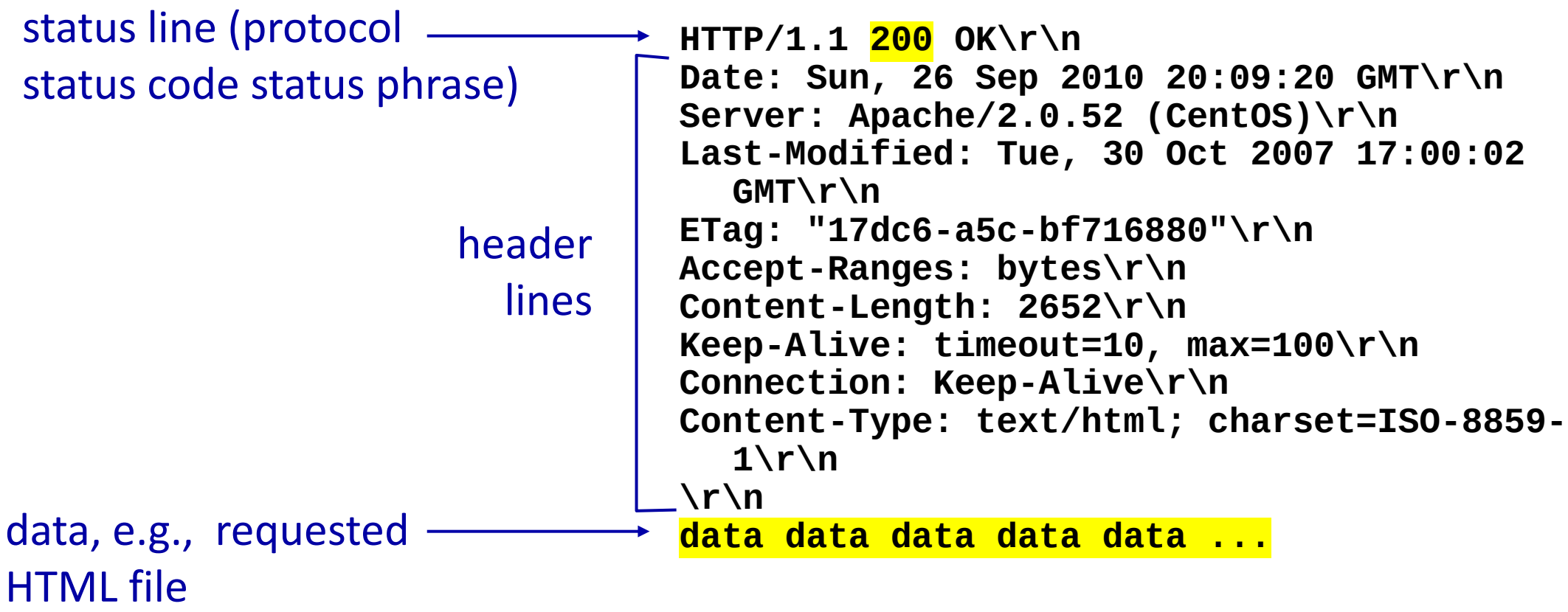
PUT method:

- uploads new file (object) to server
- ***completely replaces file*** that exists at specified URL with content in entity body of PUT HTTP request message

Summary of HTTP request messages

METHOD	DESCRIPTION
GET	Most common initial request to retrieve and render a resource (webpage)
HEAD	Same as GET, without any data in the response body
POST	Method to submit data to a web server, often used to trigger server-side functionality to process input data
PUT	Similar to POST, but submits data to overwrite the specified resource
DELETE	Deletes the specified resource
TRACE	Generates an echo of the received request; useful in debugging and auditing
OPTIONS	Returns the HTTP methods that the specified web server supports for a specific URL
CONNECT	CONNECT is used to establish a tunnel to a server through an HTTP proxy. Once the tunnel is created, raw data flows between the client and the target server; increased security
PATCH	Applies partial modifications to the specified resource

HTTP response message



HTTP response status codes

- Status code appears in 1st line in server-to-client response message.
- **Some sample status codes:**

200 OK: request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

304 Not Modified: there is no need to retransmit the requested resources

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported